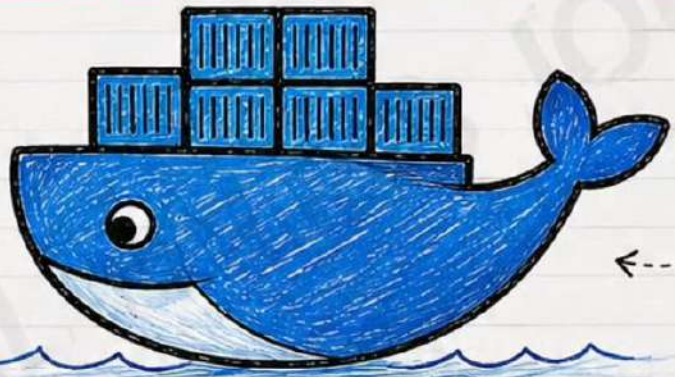
DOCKER

50 INTERVIEW QUESTIONS

50 Q&A

YOU MUST KNOW IN 2026

BEGINNER → ADVANCED → PRODUCTION



COMMANDS | DOCKERFILE | NETWORKING | TROUBLESHOOTING | PRODUCTION

DON'T JUST MEMORIZE COMMANDS.

Comment "Docker"

- ✓ PRACTICAL & PRODUCTION FOCUSED
- ✓ TROUBLESHOOTING SCENARIOS
- ✓ BEGINNER TO ADVANCED DEVOPS ENGINEERS

GLOBAL DEVOPS JOURNEY

By Sunke Nagarani

DOCKER

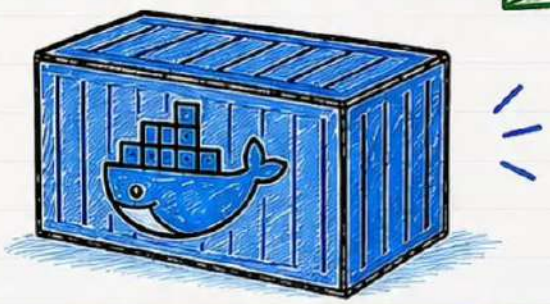
50 INTERVIEW QUESTIONS

Q1-Q5

1. DOCKER FUNDAMENTALS

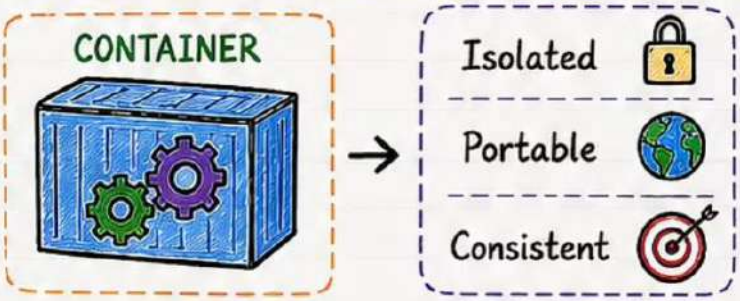
1 What is Docker?

→ Docker is a containerization platform that allows you to package, ship and run applications consistently.



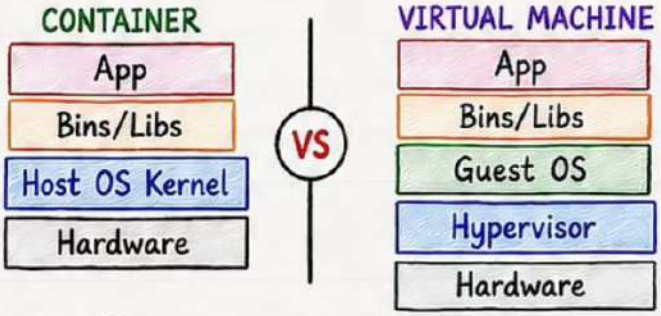
2 What is a Container?

→ A container is a lightweight, standalone and executable package that includes everything needed to run an application.



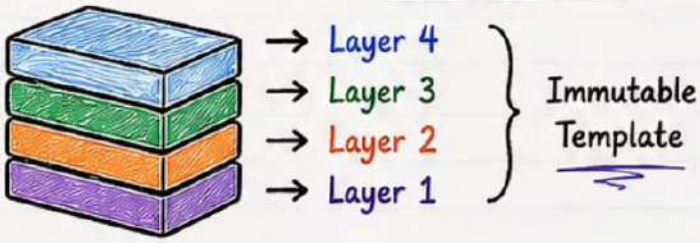
3 Container vs VM?

→ Containers share the host OS kernel, whereas VMs include a full guest OS for each instance.



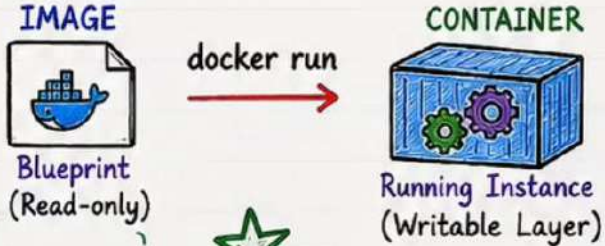
4 What is a Docker Image?

→ A Docker image is a read-only template containing application code, runtime, libraries, dependencies and other files.



5 Image vs Container?

→ Image → Blueprint (Read-only)
→ Container → Running Instance (Writable)





Key Takeaway: Image is the recipe, Container is the running food!





GLOBAL DEVOPS JOURNEY

By Sunke Nagarani



DOCKER

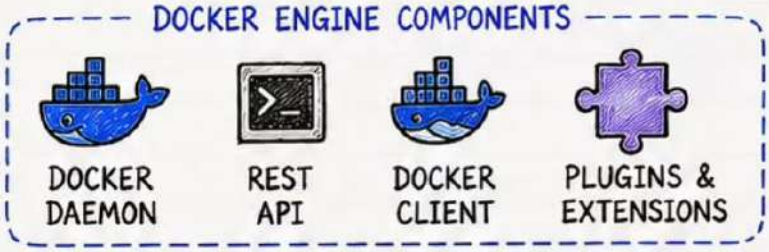
50 INTERVIEW QUESTIONS

Q6-Q10

2. DOCKER ARCHITECTURE & CORE CONCEPTS

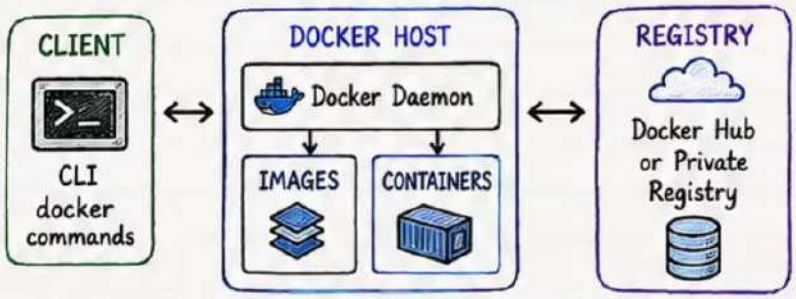
6 What is Docker Engine?

→ Docker Engine is the core technology that builds, runs and manages containers.



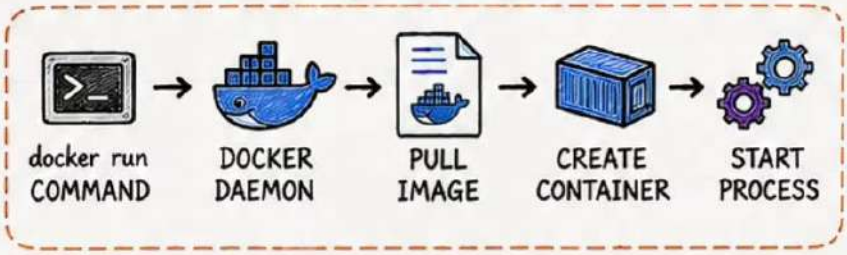
7 Explain Docker Architecture.

→ Docker uses a client-server architecture.



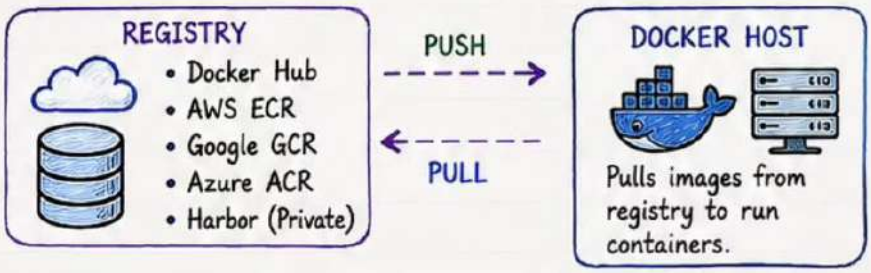
8 What happens when you run docker run?

1. Docker client sends request to daemon
2. Daemon pulls image (if not present)
3. Daemon creates a container
4. Allocates resources (CPU, Memory, Network)
5. Starts the container process



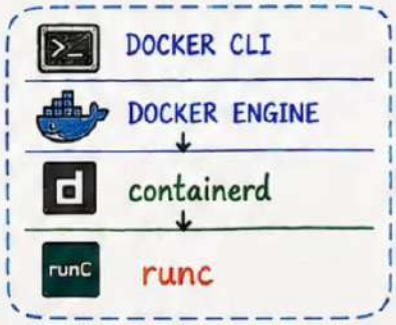
9 What is a Container Registry?

→ A registry stores and distributes container images. Examples: Docker Hub, ECR, GCR, ACR, Harbor.



10 Docker vs containerd vs runc?

TOOL	ROLE
Docker	Platform for building, shipping and managing containers.
containerd	Core container runtime used by Docker & Kubernetes.
runc	Low-level CLI to run containers according to OCI specs.





By Sunke Nagarani

DOCKER
50 INTERVIEW QUESTIONS

Q11-Q20

3. DOCKERFILE & IMAGES

11 What is a Dockerfile?

→ A text file with instructions to build a Docker image.

12 Explain FROM, RUN, COPY, WORKDIR & EXPOSE.

→ FROM – base image, RUN – execute commands, COPY – copy files, WORKDIR – set working directory, EXPOSE – document ports.

13 CMD vs ENTRYPOINT?

→ ENTRYPOINT sets the main command. CMD provides default arguments.

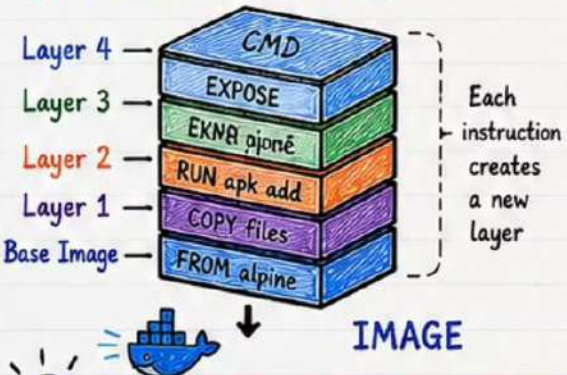
14 COPY vs ADD?

→ COPY copies files from context. ADD can also extract archives and fetch from URLs.

15 What are Image Layers?

→ Images are built in layers. Each instruction creates a new layer.

IMAGE LAYERS



Remember:
Good Dockerfile =
- ✓ Small Image
- ✓ Fast Build
- ✓ Secure
- ✓ Easy to Maintain

DOCKERFILE INSTRUCTIONS

FROM	Defines base image
RUN	Executes command & creates a layer
COPY	Copies files/folders to image
WORKDIR	Sets working directory
EXPOSE	Documents the port the app listens on
CMD	Default command (can be overridden)
ENTRYPOINT	Configures a container that will run as an executable

```
Dockerfile
FROM nginx:alpine
WORKDIR /app
COPY . /app
RUN apk add --no-cache
    curl
EXPOSE 80
CMD ["nginx", "-g",
    "daemon off;"]
```

16 How does Build Cache work?

→ Docker reuses unchanged layers from previous builds to speed up the build.



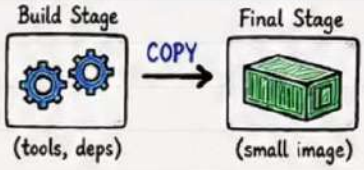
17 What is .dockerignore?

→ It excludes files & directories from the build context to reduce image size and improve build performance.

```
.dockerignore
node_modules/
.git
*.log
dist/
temp/
```

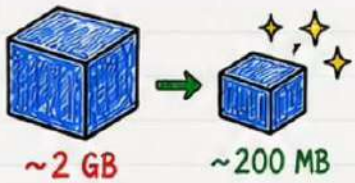
18 What is Multi-stage Build?

→ Use multiple FROM instructions. Copy only required files to final image.



19 How do you reduce image size?

→ Use small base images, multi-stage builds, .dockerignore, clean up packages, combine RUN commands.



20 Why avoid 'latest' tag?

→ It is mutable and can change anytime. Always use specific version or digest.



By Sunke Nagarani

DOCKER
50 INTERVIEW QUESTIONS

Q21-Q26

4. NETWORKING

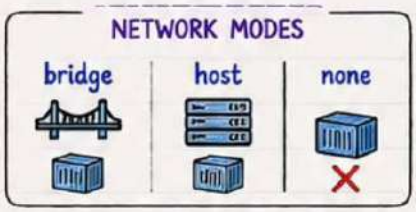
CONTAINERS COMMUNICATE USING NETWORKS

21 How does Docker networking work?

→ Docker creates isolated networks for containers. Containers on the same network can communicate.

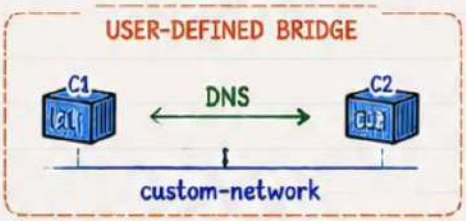
22 Bridge vs Host vs None?

→ Bridge – Default network isolation.
Host – Uses host networking stack.
None – No network connectivity.



23 What is a user-defined bridge?

→ A custom bridge network that provides better isolation and DNS name resolution between containers.

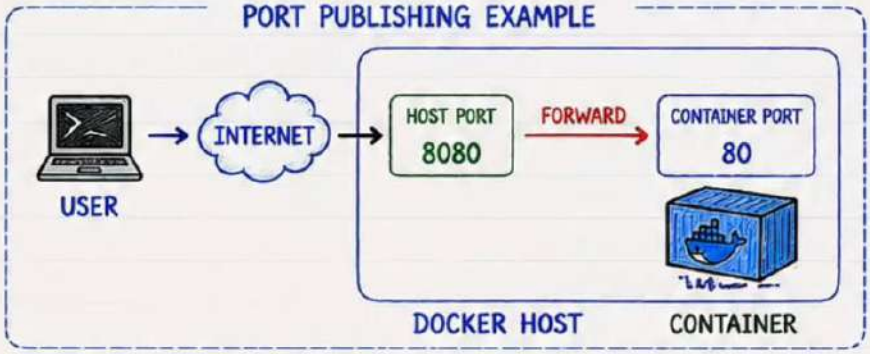


24 How do containers communicate?

→ Containers on the same network can talk to each other using container name (DNS).

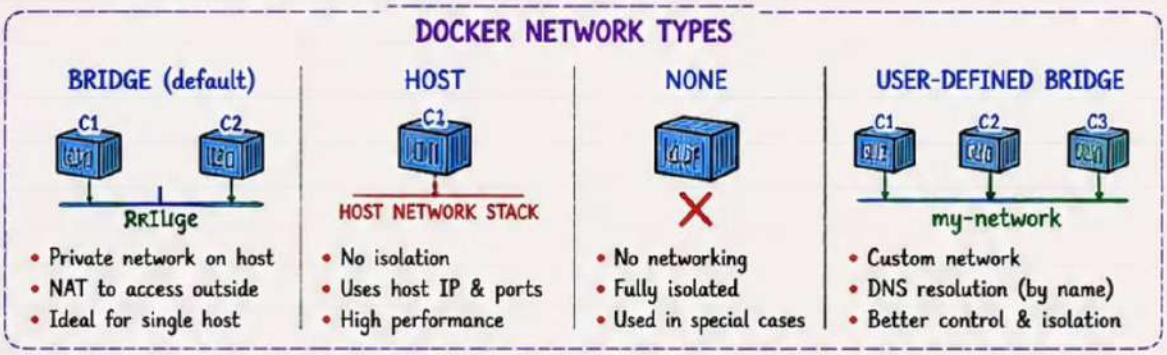
25 What does -p 8080:80 mean?

→ It maps host port 8080 to container port 80. Traffic to host:8080 will be forwarded to container:80.



26 EXPOSE vs Published Ports?

→ EXPOSE documents the port the app listens on.
→ Published ports (-p) make the port accessible from outside the container.



QUICK TIPS

- ✓ Use user-defined network for better communication.
- ✓ Use -p only when accessing from outside host.
- ✓ Use container name, not IP.

By Sunke Nagarani

DOCKER
50 INTERVIEW QUESTIONS

Q27-Q33

5. STORAGE & SECURITY BASICS

☆ STORAGE ☆

27 What is a Docker Volume?

→ Docker-managed storage that persists even if container is deleted.

28 Volume vs Bind Mount?

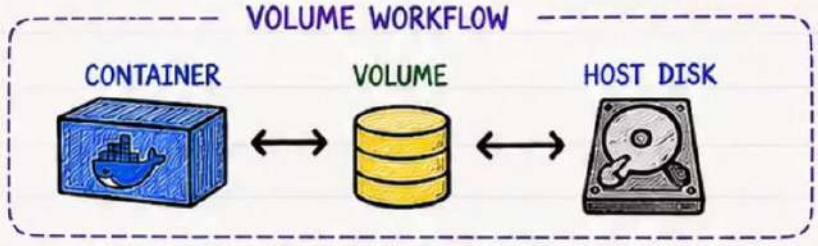
VOLUME	BIND MOUNT
Managed by Docker	Mapped to host path
Stored in Docker host	Depends on host
Portable across hosts	Not portable

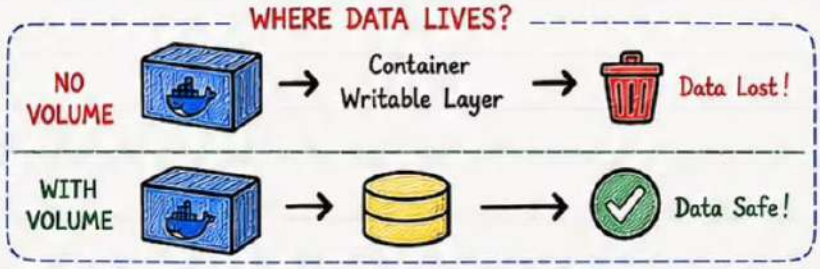
29 What happens to container data after deletion?

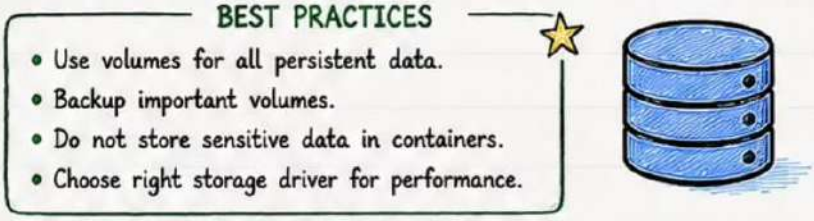
→ Writable layer is removed. Use volumes to persist data.

30 How do you persist database data in Docker?

→ Use named volumes or bind mounts. Never store critical data in container.



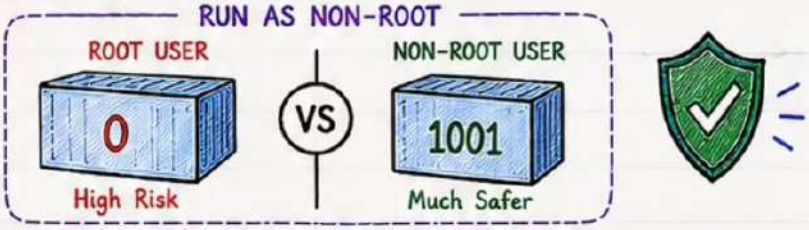




☆ SECURITY ☆

31 Why shouldn't containers run as root?

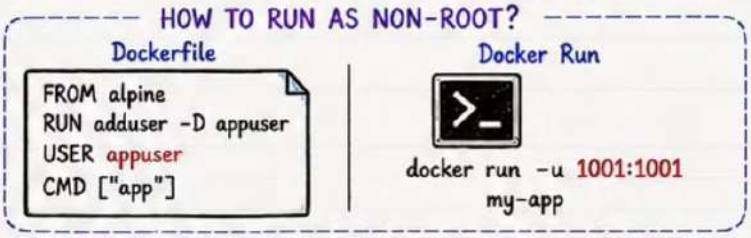
→ If container is compromised, attacker gets root access inside container.



32 How do you run container as non-root user?

→ Use USER instruction in Dockerfile or `--user` flag.

```
# Dockerfile
USER appuser
```



33 Why should secrets never be stored in images?

→ Images are shared, cached and live in registries. Secrets can be exposed.



By Sunke Nagarani

DOCKER

50 INTERVIEW QUESTIONS

Q34-Q38

6. SECURITY & OPTIMIZATION

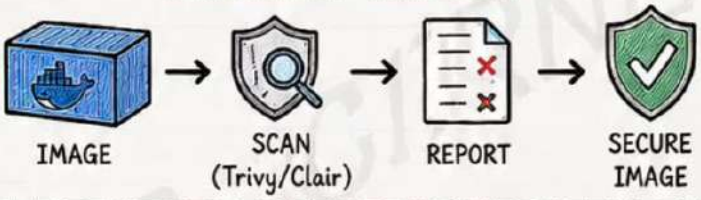
☆ SECURE CONTAINERS = SECURE PRODUCTION ☆

34 How do you scan Docker images for vulnerabilities?

→ Use tools like Trivy, Clair, Grype, or Docker Scout.



VULNERABLITY SCAN FLOW



35 Why is --privileged dangerous?

→ It gives container extended Linux capabilities. It can break isolation and harm the host.



AVOID --PRIVILEGED



- Bypasses many kernel restrictions
- Can access host devices & filesystem
- High risk of breakout attacks

36 How do CPU & memory limits work?

→ Docker uses cgroups to limit resources so containers don't consume all host resources.



LIMIT EXAMPLE

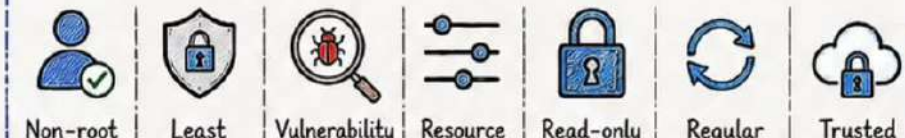
CPU LIMIT	MEMORY LIMIT	RESULT
--cpus 2	-m 512m	✓ Stable performance ✓ No resource starvation ✓ Better multi-tenancy

37 How do you secure Docker in production?

→ Use least privilege

- Run as non-root
- Scan images regularly
- Limit resources
- Use read-only file system
- Keep Docker & base images updated
- Use trusted registries

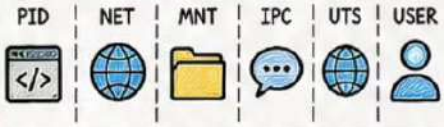
PRODUCTION SECURITY CHECKLIST



38 What are Linux namespaces & cgroups, and how does Docker use them?

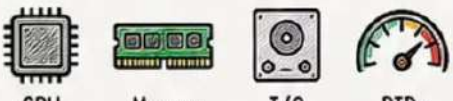
→ Namespaces provide process, network, mount isolation. cgroups limit CPU, memory, I/O usage.

NAMESPACES (Isolation)



Each container sees its own isolated view.

CGROUPS (Resource Control)



Limit and monitor resource usage.



KEY TAKEAWAY
Security is not a feature, it's a mindset.



GOOD PRACTICE TODAY
SAFE PRODUCTION TOMORROW

REMEMBER
A secure container builds a secure platform.





GLOBAL DEVOPS JOURNEY

By Sunke Nagarani



DOCKER
50 INTERVIEW QUESTIONS

Q39–Q45

8. TROUBLESHOOTING COMMON ISSUES

☆ PRODUCTION ISSUES ARE REAL INTERVIEW QUESTIONS ☆

39 Container exits immediately?

→ Check logs to find the root cause.

`docker logs <container_id>`



40 Container keeps restarting?

→ Check restart policy, logs & resources.

`docker inspect <container_id>`



41 App works in container but not in browser?

→ Check ports, firewall & network.

`docker port <container_id>`



42 Containers cannot communicate?

→ Check network mode, DNS & firewall.

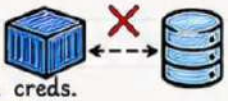
`docker network ls`



43 Container can't reach database?

→ Check DB URL, network, DNS & creds.

`docker exec -it <container> sh`



44 Docker disk usage = 100%?

→ Clean unused objects & prune.

```
docker system df
docker system prune -a
```



45 Container CPU/Memory = 100%?

→ Check resource usage & limit the container.

`docker stats`



TROUBLESHOOTING PROCESS

1. **IDENTIFY SYMPTOM** — What is the issue?
2. **CHECK LOGS** — `docker logs <container>`
3. **INSPECT CONTAINER** — `docker inspect <container>`
4. **CHECK RESOURCES** — CPU, Memory, Disk, Network
5. **CHECK NETWORK** — docker network, ping, DNS
6. **FIND ROOT CAUSE** — Fix the issue & verify

USEFUL DOCKER COMMANDS

View running containers	`docker ps`
View all containers	`docker ps -a`
Container logs	`docker logs <id>`
Inspect container	`docker inspect <id>`
Execute inside container	`docker exec -it <id> sh`
Resource usage	`docker stats`
Network list	`docker network ls`
Prune unused data	`docker system prune -a`
Disk usage	`docker system df`

TROUBLESHOOTING IS NOT JUST COMMANDS, IT'S ALL ABOUT LOGIC, PATIENCE & EXPERIENCE!

REAL ENGINEERS FIX ISSUES, NOT JUST DEPLOY APPS!

DOCKER
50 INTERVIEW QUESTIONS

Q46-Q50

9. PERFORMANCE & BEST PRACTICES

☆ SPEED + EFFICIENCY = BETTER PRODUCTION SYSTEMS ☆

46 How do you make Docker containers run faster?

→ Optimize images, reduce layers, use caching, allocate proper resources.

SPEED OPTIMIZATION TIPS

- Use Small Base Images (alpine)
- Reduce Image Layers
- Clean Up Images & Containers
- Allocate Proper CPU & Memory Limits
- Use Multi-stage Builds

47 How do you reduce Docker image size?

→ Use .dockerignore, multi-stage builds, remove unnecessary files & packages.

REDUCE IMAGE SIZE

.dockerignore (Exclude files) → Multi-stage Builds → Remove Unused Packages → Use Specific Versions → Smaller Image = Faster Deploy

48 How do you clean up unused Docker resources?

→ Remove unused containers, images, volumes & networks regularly.

CLEANUP COMMANDS

Stop all containers	Remove all containers	Remove unused images	Remove unused volumes	Remove unused networks
`docker stop $(docker ps -aq)`	`docker rm -f $(docker ps -aq)`	`docker image prune -a`	`docker volume prune`	`docker network prune`

49 What are Docker logging options?

→ Use json-file, syslog, fluentd, gelf, awslogs based on need.

COMMON LOGGING DRIVERS

- json-file (Default)
- syslog
- fluentd
- gelf
- awslogs

50 What are Docker best practices?

→ Follow best practices for secure, reliable & scalable containerized applications.

DOCKER BEST PRACTICES

- ✓ Use official & trusted base images
- ✓ Keep images small & simple
- ✓ Run as non-root user
- ✓ Scan images for vulnerabilities
- ✓ Use .dockerignore file
- ✓ Remove unused images & containers
- ✓ Limit container resources (CPU/Memory)
- ✓ Use read-only file system if possible
- ✓ Use secrets management
- ✓ Keep Docker & base images updated
- ✓ Use logging & monitoring
- ✓ Follow least privilege principle

GOOD DOCKER ENGINEERS
Write efficient Dockerfiles, Build secure images & Run reliable containers!

Tip: Small Images Fast Deploys Happy Teams!

FOCUS ON BEST PRACTICES TODAY FOR A SCALABLE & SECURE PRODUCTION TOMORROW!

DOCKER

50 INTERVIEW QUESTIONS

Q51-Q55

CAN YOU ANSWER ALL 50?

1. FUNDAMENTALS

1-10

- Docker concepts
- Architecture
- Images vs Containers
- Commands
- Real-world use

2. DOCKERFILE

11-20

- Instructions
- Best practices
- Build process
- Optimization
- Multi-stage builds

3. NETWORKING/ STORAGE

21-30

- Networks
- Ports & mapping
- Volumes
- Bind mounts
- Data persistence

4. SECURITY

31-38

- Image security
- Least privilege
- Capabilities
- Secrets
- Scanning tools

5. TROUBLESHOOTING + PRODUCTION

39-50

- Logs & events
- Failures
- Performance
- Monitoring
- Real-world fixes

INTERVIEW ANSWER FORMULA

1. WHAT?
2. HOW?
3. REAL EXAMPLE
4. WHAT CAN FAIL?
5. HOW TO TROUBLESHOOT?

DON'T JUST SAY: "I KNOW DOCKER."

PROVE YOU CAN RUN IT IN PRODUCTION.

Comment "DOCKER" for the complete 50 Q&A

GLOBAL DEVOPS JOURNEY

By Sunke Nagarani